

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático II (TP II)

BCC203– Estruturas de Dados II

Beatriz Aparecida Coelho Miguel, Eduardo Henrique Fernandes Marques, e Maria
Clara Maia Moreira

Professor: Guilherme Tavares de Assis

Ouro Preto
13 de março de 2025

Sumário

1	Introdução	1
1.1	Seleção por Substituição	1
1.2	Intercalação balanceada de vários caminhos (2f fitas)	1
1.3	Intercalação balanceada de vários caminhos ($f+1$ fitas)	2
1.4	<i>Quicksort</i> externo	2
2	Desenvolvimento	3
2.1	Seleção por Substituição	3
2.2	Intercalação balanceada de vários caminhos (2f fitas)	4
2.3	Intercalação balanceada de vários caminhos ($f+1$ fitas)	6
2.4	<i>Quicksort</i> externo	8
3	Instruções de Compilação	9
4	Conclusão	9

Lista de Figuras

1	Seleção por substituição - 1	3
2	Seleção por substituição - 2	3
3	Seleção por substituição - 3	3
4	2F Caminhos - 1	4
5	2F Caminhos - 2	5
6	2F Caminhos - 3	5
7	$f + 1$ Caminhos - 1	6
8	$f + 1$ Caminhos - 2	6
9	$f + 1$ Caminhos - 3	7
10	QuickSort - 1	8
11	QuickSort - 2	8
12	QuickSort - 3	8

1 Introdução

O trabalho tem como objetivo a implementação e análise de três métodos de ordenação externa, considerando sua complexidade de desempenho em diferentes aspectos. A ordenação externa é essencial para lidar com grandes volumes de dados que não podem ser completamente armazenados na memória principal, exigindo novas estratégias de manipulação em memória secundária. Os métodos a serem analisados são: (1) Intercalação balanceada de vários caminhos ($2f$ fitas) com seleção por substituição na geração de blocos ordenados, (2) Intercalação balanceada de vários caminhos ($f+1$ fitas) também com seleção por substituição na geração de blocos ordenados, e (3) Quicksort externo. A análise de desempenho será realizada por meio de experimentos que avaliarão os seguintes aspectos: o número de transferências (leitura) de registros da memória externa para a memória interna, o número de transferências (escrita) de registros da memória interna para a memória externa, o número de comparações entre valores do campo de ordenação e o tempo total de execução. Para isso, utilizaremos a base de dados "PROVAO.TXT", contendo informações de 471.705 alunos que participaram do "Provão" de 2003, permitindo a observação do comportamento dos métodos diante de um grande conjunto de registros. Essa análise possibilitará uma melhor compreensão das diferenças entre os métodos de ordenação externa estudados e seu impacto no desempenho, considerando o aumento do volume de dados e as restrições de memória disponíveis.

Configurações do dispositivo de teste:

- Processador: Ryzen 5
- Memória RAM: 8 GB
- Sistema Operacional: Windows

1.1 Seleção por Substituição

A seleção por substituição é uma técnica eficiente utilizada para ordenação externa de grandes volumes de dados, onde a memória principal não é suficiente para armazenar todas as informações ao mesmo tempo. Esse método é amplamente empregado em sistemas que lidam com grandes bases de dados, como bancos de dados e processamento de arquivos extensos. O objetivo principal é organizar os registros de forma sequencial em arquivos intermediários chamados de fitas, facilitando a posterior intercalação para gerar um arquivo final ordenado.

A principal vantagem desse método está na sua capacidade de gerar longas sequências ordenadas em um único passo de leitura do arquivo de entrada, reduzindo o número de operações de escrita e leitura em disco, que são geralmente custosas em termos de tempo. A técnica utiliza um heap mínimo para armazenar os registros temporários e garantir que os menores elementos sejam processados primeiro, mantendo um fluxo contínuo de ordenação. Isso resulta em um algoritmo eficiente para manipulação de grandes quantidades de dados sem a necessidade de armazenar todo o conjunto em memória.

O código implementa essa técnica para ordenar registros armazenados em um arquivo de texto. Esse processo permite que os registros sejam organizados de forma eficiente, garantindo a correta estruturação dos dados e permitindo sua posterior intercalação para formar um arquivo completamente ordenado.

1.2 Intercalação balanceada de vários caminhos ($2f$ fitas)

A intercalação utilizando $2f$ fitas é uma técnica de ordenação externa que amplia a eficiência do processo de fusão de blocos ordenados ao aproveitar um número maior de fitas disponíveis. Inicialmente, o arquivo de entrada é dividido em blocos que cabem na memória interna, e cada bloco é ordenado individualmente. Esses blocos ordenados são então distribuídos de forma equilibrada entre f fitas, que funcionam como fitas de entrada. As outras f fitas restantes são reservadas como fitas de saída, onde os resultados intermediários da intercalação serão armazenados.

O processo de intercalação ocorre em passos sequenciais. Em cada passo, os primeiros registros de cada bloco nas fitas de entrada são lidos e comparados. O menor registro entre eles é selecionado e gravado em uma das fitas de saída. Esse registro é então substituído pelo próximo registro do mesmo bloco da fita de entrada, e o processo se repete até que todos os registros daquele bloco tenham sido

processados. Conforme os registros são gravados nas fitas de saída, os blocos são intercalados de forma ordenada.

Após a conclusão de um passo, as fitas de entrada e saída trocam de função. As fitas que antes eram de saída passam a ser as novas fitas de entrada para o próximo passo, e vice-versa. Esse ciclo de alternância entre fitas de entrada e saída continua até que todos os registros estejam completamente ordenados em uma única fita. A utilização de $2f$ fitas permite uma organização mais eficiente dos dados, reduzindo o número de passos necessários para concluir a ordenação e otimizando o uso das fitas disponíveis. Essa técnica é especialmente vantajosa quando se lida com grandes volumes de dados, onde a memória interna é insuficiente para armazenar todos os dados simultaneamente.

1.3 Intercalação balanceada de vários caminhos ($f+1$ fitas)

A intercalação de $f+1$ caminhos é uma técnica eficiente empregada na ordenação de grandes volumes de dados em sistemas de ordenação externa, especialmente quando a memória interna não é suficiente para armazenar todos os dados simultaneamente. Esse método divide o arquivo de entrada em blocos que cabem na memória, ordena-os internamente e os armazena em mídias externas, como fitas magnéticas.

Na fase de intercalação, a técnica utiliza apenas $f+1$ fitas para realizar a fusão dos blocos ordenados. O processo inicia-se com a leitura do primeiro registro de cada bloco armazenado nas fitas. O menor registro é removido e gravado em uma fita de saída, enquanto um novo registro do mesmo bloco substitui o que foi retirado. Esse procedimento se repete até que todos os registros tenham sido processados.

Uma das principais vantagens dessa abordagem é a redistribuição dos blocos entre as fitas após cada passada de intercalação. Diferente de outras técnicas que exigem um número de fitas proporcional à quantidade de blocos, a intercalação de $f+1$ caminhos reduz a necessidade de fitas, otimizando recursos. Após cada passada, os blocos ordenados são reorganizados entre as fitas para dar continuidade ao processo de ordenação de maneira eficiente.

Embora o número de passadas varie conforme o tamanho do arquivo e a capacidade da memória interna, essa técnica se destaca por equilibrar o uso dos recursos e minimizar o custo de execução do algoritmo de ordenação externa. Assim, a intercalação de $f+1$ caminhos oferece uma solução eficiente e escalável para manipulação de grandes volumes de dados.

1.4 *Quicksort* externo

O *Quicksort* externo, é um método de ordenação externa baseado no método de mesmo nome comumente usado para ordenar dados em memória principal. De maneira análoga, um pivô é escolhido dentro do arquivo que deseja ser ordenado e, assim, também são definidos dois "subarquivos" (um que contenha itens menores relativos ao pivô, e um que contenha itens maiores relativos ao pivô). Os subarquivos são, na realidade, apenas delimitações (dentro do mesmo arquivo) cada vez menores de uma certa quantidade de dados que é ordenada regressivamente, assim como no algoritmo original. Diferenciando-se do *Quicksort* original, no entanto, a construção do pivô no *Quicksort* externo é menos arbitrária e mais dinâmica, além de o próprio pivô ser constituído de não apenas um item, mas vários, a depender do tamanho disponível na memória principal. Neste trabalho, em específico, foi considerada um vetor de 10 posições na memória principal. O arquivo a ser ordenado é percorrido do início ao fim, e do fim ao início por dois ponteiros, fazendo as leituras e comparações necessárias e, assim, constituindo o pivô, que ao final da partição, conterá os itens intermediários (em valor) de todo o arquivo, e tais itens já são posicionados de forma ordenada em relação aos outros itens do arquivo, e entre si. As partições são realizadas recursivamente e ocorrem até que o tamanho dos subarquivos seja inválido.

2 Desenvolvimento

2.1 Seleção por Substituição

Seleção por Substituição - Ordem Crescente				
Número de Registros	Tempo de Execução	Transferências por Leitura	Transferências por escrita	Comparações
100	0.000515 s	100	100	3 510
200	0.000734 s	200	200	7 410
2000	0.003644 s	2 000	2 000	77 610
20000	0.022442 s	20 000	20 000	779 610
200000	0.367315 s	200 000	200 000	7 819 964
471705	0.671614 s	471 705	471 705	18 459 639

Figura 1: Seleção por substituição - 1

Seleção por Substituição - Ordem Decrescente				
Número de Registros	Tempo de Execução	Transferências por Leitura	Transferências por escrita	Comparações
100	0.014296 s	100	100	3 175
200	0.001171 s	200	200	6 800
2000	0.004285 s	2 000	2 000	72 828
20000	0.026816 s	20 000	20 000	764 869
200000	0.317858 s	200 000	200 000	7 752 896
471705	0.711430 s	471 705	471 705	18 309 213

Figura 2: Seleção por substituição - 2

Seleção por Substituição - Ordem Aleatória				
Número de Registros	Tempo de Execução	Transferências por Leitura	Transferências por escrita	Comparações
100	0.010578 s	100	100	3 505
200	0.001250 s	200	200	7 191
2000	0.003850 s	2 000	2 000	74 805
20000	0.028289 s	20 000	20 000	750 385
200000	0.333415 s	200 000	200 000	7 472 305
471705	0.719052 s	471 705	471 705	17 620 979

Figura 3: Seleção por substituição - 3

O processo começa com a leitura dos registros do arquivo de entrada, onde cada linha contém informações como inscrição, nota, estado, cidade e curso. Esses registros são armazenados em um vetor de tamanho fixo, que funciona como um buffer temporário para processamento. Em seguida, o algoritmo constrói um heap, garantindo que o menor elemento esteja sempre na raiz. Isso facilita a extração eficiente do menor registro, que é removido do heap e escrito em um dos arquivos de saída, representados por fitas magnéticas simuladas.

Após a extração, um novo registro é lido do arquivo de entrada e inserido no heap, desde que o número máximo de leituras ainda não tenha sido atingido. Caso o novo registro seja menor que o registro extraído anteriormente, ele é marcado para ser processado em um próximo bloco. O algoritmo verifica constantemente se todos os registros do buffer foram marcados. Quando isso ocorre, um bloco é finalizado e uma linha em branco é adicionada ao arquivo de fita, indicando a separação entre diferentes blocos ordenados. Em seguida, os registros do buffer são desmarcados, permitindo que um novo ciclo de ordenação seja iniciado utilizando outra fita para gravação, garantindo um balanceamento na distribuição dos registros entre os arquivos.

Quando os registros do arquivo de entrada terminam, o algoritmo continua extraindo os elementos remanescentes do heap e gravando-os nas fitas, assegurando que todos os registros sejam processados em blocos ordenados. Cada bloco é separado por uma linha em branco, facilitando a posterior intercalação dos dados. Essa abordagem permite lidar com grandes volumes de informações sem comprometer a memória, garantindo eficiência no processamento.

Os resultados obtidos com a seleção por substituição demonstram um desempenho consistente conforme a quantidade de dados aumenta. Para arquivos com 100 registros, o tempo de execução foi baixo, de aproximadamente 0.000515 segundos para arquivos ordenados de forma crescente. O número de comparações e transferências foi bem controlado, com 3.510 comparações e 100 transferências de leitura e escrita.

Com o aumento do número de registros, o tempo de execução cresceu de forma proporcional, mas controlada. Para arquivos com 200.000 registros, o tempo de execução foi de cerca de 0.367145 segundos, com 7.819.964 comparações e 200.000 transferências. Para arquivos com 471.705 registros, o desempenho do algoritmo se manteve eficiente, com um tempo de execução de 0.676145 segundos, demonstrando sua capacidade de lidar com grandes volumes de dados.

Quando os dados foram ordenados de forma decrescente, os resultados foram similares aos obtidos com arquivos ordenados de forma crescente, embora com tempos ligeiramente maiores em alguns casos. O tempo de execução apresentou diferenças mínimas, e a performance do algoritmo não foi significativamente impactada pela ordem inicial dos registros. Isso sugere que o método é robusto e eficiente independentemente da distribuição inicial dos dados.

Para arquivos com registros aleatórios, o comportamento do algoritmo se manteve consistente, com tempos de execução aumentando gradualmente conforme o número de registros crescia. O tempo variou de 0.000578 segundos para 100 registros até 0.719052 segundos para 471.705 registros. O padrão de crescimento no número de comparações e transferências refletiu a forma como o algoritmo lida com a complexidade do processamento dos dados.

No geral, os resultados indicam que o algoritmo é eficaz para grandes volumes de dados, mantendo tempos de execução dentro dos limites, mesmo para arquivos com milhares de registros. Ele lida bem com diferentes distribuições de dados e é capaz de realizar a ordenação de forma eficiente sem sobrecarregar a memória. Esse desempenho torna o algoritmo adequado para aplicações que exigem o processamento de grandes quantidades de informações.

2.2 Intercalação balanceada de vários caminhos (2f fitas)

Intercalação balanceada de vários caminhos (2f fitas) - Ordem Crescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.000593 s	100	100	100
200	0.000689 s	200	200	200
2000	0.002424 s	2 000	2 000	2 000
20000	0.021681 s	20 000	20 000	20 000
200000	0.257946 s	200 000	200 000	200 000
471705	0.618717 s	471 705	471 705	471 705

Figura 4: 2F Caminhos - 1

Intercalação balanceada de vários caminhos (2f fitas) - Ordem Decrescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.007582 s	107	100	426
200	0.002394 s	416	400	1 189
2000	0.005304 s	4 063	4000	18696
20000	0.080250 s	60 162	60 000	236 525
200000	0.663473 s	600 428	600 000	2 857 326
471705	1.707526 s	1 415 859	1 415 115	7 526 954

Figura 5: 2F Caminhos - 2

Intercalação balanceada de vários caminhos (2f fitas) - Ordem Aleatória				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.006594 s	105	100	226
200	0.001459 s	413	400	740
2000	0.006482 s	4 107	4 000	10 208
20000	0.060920 s	61 064	60 000	143 347
200000	0.918200 s	810 782	800 000	1 809 922
471705	2.887654 s	2 383 736	2 358 525	5 192 796

Figura 6: 2F Caminhos - 3

O código desenvolvido tem como principal função a intercalação de fitas ordenadas utilizando $2f$ caminhos. A função responsável por essa tarefa é a `intercalarFitasOrdenadas2F`, que representa o centro do algoritmo de ordenação externa. O objetivo principal dessa função é unir blocos de registros provenientes de duas fitas ordenadas, resultando em uma fita de saída única e ordenada. A função começa inicializando as fitas de entrada e as fitas de saída. Para cada uma das fitas de entrada, o código aloca memória para armazenar os blocos de registros que serão lidos. Cada bloco lido é armazenado em um array dinâmico, e o número de registros lidos de cada fita é mantido em um vetor para controle. O algoritmo percorre as fitas de entrada e, a cada leitura de um bloco, a função `lerBloco` é chamada para preencher os blocos com registros. Essa leitura é feita até que todos os blocos sejam processados.

Uma vez que os blocos foram lidos de todas as fitas, a função inicia o processo de intercalação propriamente dito. A ideia principal é buscar, entre os registros dos blocos lidos, o menor valor para ser escrito na fita de saída. Para isso, a função compara o primeiro registro de cada bloco e, se o registro for o menor, ele é escrito na fita de saída. Em seguida, o índice daquele bloco é incrementado, e a busca pelo próximo menor valor continua até que todos os registros de todos os blocos sejam processados. A principal vantagem dessa abordagem é o uso da memória e dos recursos de forma eficiente. Como o algoritmo trabalha com blocos de registros, é possível lidar com grandes volumes de dados sem sobrecarregar a memória. Além disso, o número de passagens pelas fitas é minimizado, o que contribui para a eficiência geral do processo de ordenação. O algoritmo, ao intercalar as fitas ordenadas, garante que, ao final do processo, todos os dados estarão ordenados e prontos para serem redistribuídos entre as fitas de entrada.

No final da execução da função `intercalarFitasOrdenadas2F`, os dados ordenados são gravados na fita de saída, e todas as fitas de entrada são fechadas e liberadas da memória. A ordenação é concluída quando resta apenas um bloco na fita de saída, sinalizando que todos os dados foram ordenados com sucesso. Esse método de intercalação de 2 caminhos é uma solução eficaz para a ordenação de grandes volumes de dados que não cabem inteiramente na memória RAM, aproveitando ao máximo o uso do armazenamento externo e mantendo um bom desempenho ao longo da execução.

As fitas de entrada e saída são inicializadas com base no número da iteração. A cada iteração,

as fitas de entrada e saída alternam entre as fitas 1 e 11, garantindo que os dados sejam intercalados corretamente. A função lerBloco é responsável por ler os registros das fitas de entrada e armazená-los em blocos ordenados. Esses blocos são mantidos em memória para que possam ser comparados e intercalados. O algoritmo compara os registros dos blocos lidos e escreve o menor registro na fita de saída. Esse processo é repetido até que todos os registros de todos os blocos sejam processados. Após a intercalação, as fitas de entrada são fechadas e a memória alocada para os blocos é liberada. A função verifica se o processo de intercalação foi concluído com sucesso e retorna o número da iteração para indicar qual fita contém os dados ordenados.

O método de intercalação com 2 fitas é uma solução eficiente e eficaz para a ordenação de grandes volumes de dados que não cabem inteiramente na memória RAM. Ao aproveitar ao máximo o uso do armazenamento externo e manter um bom desempenho ao longo da execução, essa abordagem garante que todos os dados sejam ordenados corretamente e prontos para uso.

2.3 Intercalação balanceada de vários caminhos ($f+1$ fitas)

Intercalação balanceada de vários caminhos (f+1 fitas) - Ordem Crescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.001040 s	202	100	100
200	0.000952 s	402	200	200
2000	0.002516 s	4 002	2 000	2 000
20000	0.017564 s	40 002	20 000	20 000
200000	0.287056 s	400 002	200 000	200 000
471705	0.597877 s	943 412	471 705	471 705

Figura 7: $f + 1$ Caminhos - 1

Intercalação balanceada de vários caminhos (f+1 fitas) - Ordem Decrescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.008400 s	206	100	290
200	0.001069 s	409	200	977
2000	0.008438 s	8 049	4 000	23 935
20000	0.051976 s	80 131	40 000	284 234
200000	0.561289 s	800 371	400 000	4 451 531
471705	1.656105 s	2 830 905	1 415 115	10 365 336

Figura 8: $f + 1$ Caminhos - 2

Intercalação balanceada de vários caminhos (f+1 fitas) - Ordem Aleatória				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.007003 s	205	100	181
200	0.001113 s	407	200	470
2000	0.007816 s	8 060	4 000	9 952
20000	0.081466 s	120 569	60 000	133 871
200000	0.784229 s	1 205 673	600 000	1 466 645
471705	2.233772 s	3 786 900	1 886 820	4 057 377

Figura 9: $f + 1$ Caminhos - 3

O código desenvolvido tem como principal função a intercalação de fitas ordenadas, utilizando uma técnica conhecida como $f+1$ caminhos. A função responsável por essa tarefa é a *intercalarFitasOrdenadas*, que é o núcleo do algoritmo de ordenação externa. O objetivo principal dessa função é unir blocos de registros provenientes de diversas fitas ordenadas, resultando em uma fita de saída única e ordenada.

A *intercalarFitasOrdenadas* começa inicializando as fitas de entrada e a fita de saída. Para cada uma das fitas de entrada, o código aloca memória para armazenar os blocos de registros que serão lidos. Cada bloco lido é armazenado em um array dinâmico, e o número de registros lidos de cada fita é mantido em um vetor para controle. O algoritmo percorre as fitas de entrada e, a cada leitura de um bloco, a função *lerBloco* é chamada para preencher os blocos com registros. Essa leitura é feita até que todos os blocos sejam processados.

Uma vez que os blocos foram lidos de todas as fitas, a função inicia o processo de intercalação propriamente dito. A ideia principal é buscar, entre os registros dos blocos lidos, o menor valor para ser escrito na fita de saída. Para isso, a função compara o primeiro registro de cada bloco e, se o registro for o menor, ele é escrito na fita de saída. Em seguida, o índice daquele bloco é incrementado, e a busca pelo próximo menor valor continua até que todos os registros de todos os blocos sejam processados.

A principal vantagem dessa abordagem é o uso da memória e dos recursos de forma eficiente. Como o algoritmo trabalha com blocos de registros, é possível lidar com grandes volumes de dados sem sobrecarregar a memória. Além disso, o número de passagens pelas fitas é minimizado, o que contribui para a eficiência geral do processo de ordenação. O algoritmo, ao intercalar as fitas ordenadas, garante que, ao final do processo, todos os dados estarão ordenados e prontos para serem redistribuídos entre as fitas de entrada. No final da execução da função *intercalarFitasOrdenadas*, os dados ordenados são gravados na fita de saída, e todas as fitas de entrada são fechadas e liberadas da memória. A ordenação é concluída quando resta apenas um bloco na fita de saída, sinalizando que todos os dados foram ordenados com sucesso. Esse método de intercalação de $f+1$ caminhos é uma solução eficaz para a ordenação de grandes volumes de dados que não cabem inteiramente na memória RAM, aproveitando ao máximo o uso do armazenamento externo e mantendo um bom desempenho ao longo da execução.

Analisando os resultados das tabelas, a ordenação crescente foi a mais eficiente. Esse método apresentou os menores tempos de ordenação, bem como o menor número de transferências por leitura e escrita, além de um número reduzido de comparações. Isso ocorre porque os registros já estão organizados em ordem crescente, exigindo menos operações para serem intercalados corretamente.

A ordenação aleatória teve um desempenho intermediário. Como os dados não seguem um padrão específico, o algoritmo precisou realizar mais operações para ordenar os registros, aumentando o número de transferências e comparações em relação ao caso crescente. No entanto, esse aumento não foi tão expressivo quanto no pior cenário, o da ordenação decrescente.

A ordenação decrescente foi a menos eficiente, apresentando tempos de execução significativamente maiores, um número elevado de transferências e um número muito superior de comparações. Isso ocorre porque os registros estavam organizados na ordem oposta à esperada, exigindo o máximo de operações para que o algoritmo conseguisse ordená-los corretamente. Esse cenário gera um alto custo computacional, tornando a ordenação decrescente o pior caso entre os três analisados.

2.4 Quicksort externo

Quicksort externo - Ordem Crescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.000870 s	100	100	182
200	0.000781 s	200	200	382
2000	0.006307 s	2 000	2 000	3 982
20000	0.074585 s	20 000	20 000	39 982
200000	1.052391 s	299 980	299 980	1 343 481
471705	2.932178 s	731 908	731 908	3 234 487

Figura 10: QuickSort - 1

Quicksort externo - Ordem Decrescente				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.000411 s	100	100	438
200	0.001272 s	200	200	908
2000	0.009566 s	2 854	2 854	12 638
20000	0.150722 s	37 429	37 429	161 906
200000	0.949754 s	225 909	225 909	918 577
471705	2.665107 s	731 908	731 908	3 026 995

Figura 11: QuickSort - 2

Quicksort externo - Ordem Aleatória				
Número de Registros	Tempo de Ordenação	Transferências por Leitura	Transferências por escrita	Comparações
100	0.000564 s	100	100	600
200	0.003693 s	732	732	2 013
2000	0.041817 s	12 135	12 135	25 573
20000	0.555343 s	165 411	165 411	289 216
200000	8.827265 s	2 441 475	2 441 475	3 637 107
471705	18.964117 s	5 951 832	5 951 832	8 635 591

Figura 12: QuickSort - 3

O *QuickSort* externo é um algoritmo bastante vantajoso para ordenação em memória externa, uma vez que consegue lidar com um grande volume de dados realizando um número de transferências razoável e minimizando os acessos a disco.

Ao lidar com arquivos ordenados tanto crescentemente quanto decrescentemente, foi observado que o algoritmo realizou 731908 transferências tanto de escrita quanto de leitura ao ordenar os arquivos de maior extensão (471705 registros), mostrando que nos melhores casos, os números de transferências dos contextos individuais não alcançam o dobro do número de registros. Já com o arquivo inicialmente desordenado, o número de transferências de leitura e de escrita individualmente são em torno de 6 milhões, um número consideravelmente mais elevado mas que ainda não deixa de ser aceitável, já que

o número de partições realizadas com os números ordenados aleatoriamente cresce com rapidez. Uma mudança positiva que pode ser realizada para evitar que as partições em memória secundária sejam realizadas muitas vezes, é adicionar uma condicional e uma função que verificam se o "subarquivo" a ser ordenado é menor que a memória disponível na RAM, ou seja, que o tamanho da área. Em caso positivo, pode ser feita uma única leitura de tal subarquivo e ordenar os registros obtidos diretamente em memória interna. Ao ordená-los, também é possível transferi-los para a posição correta de uma só vez, ou seja, utilizando apenas uma operação de escrita. Por falta de tempo, entretanto, não foi possível implementar esse algoritmo no trabalho.

A complexidade de tempo do *QuickSort* é de $n \log n$, que apesar de ser uma boa complexidade, é fácil perceber que com um número de dados muito grande (por volta dos milhões) o algoritmo poderia ser consideravelmente mais demorado, gastando minutos para realizar a ordenação, ressaltando que em nenhum dos experimentos realizados, considerando todas as condições, o algoritmo gastou mais que 1 minuto para ser executado.

Quanto ao número de comparações, é notável que ele cresce consideravelmente de acordo com a situação inicial do arquivo, chegando a fazer cerca de três vezes mais comparações entre as chaves com os dados desordenados em relação aos dados ordenados. As comparações tanto na situação 1 quanto na situação 2 (crescente e decrescente, respectivamente) se mantêm constantes como era o esperado, ou seja, o número não muda para testes iguais realizados com o mesmo número de chaves. Como o arquivo "PROVAO.TXT" usado para os testes possui sempre a mesma "ordem aleatória" então as comparações sempre serão as mesmas para um mesmo volume de dados, logicamente. No entanto, para contextos mais gerais, o número de comparações do método pode oscilar bastante de acordo com o quanto desorganizados os dados estão.

3 Instruções de Compilação

```
gcc -Wall -o exe main.c funcoes.c selecao.c quicksort.c interf+1.c inter2f.c -I include
```

4 Conclusão

Concluímos que a escolha do método de ordenação externa depende muito do tamanho dos dados e dos recursos disponíveis. A intercalação com $f + 1$ fitas é a mais simples de implementar e funciona bem para volumes menores de dados, mas pode ficar lenta quando os dados são muito grandes, já que precisa passar várias vezes pelas fitas. Já a intercalação com $2f$ fitas é mais eficiente para grandes volumes, pois reduz o número de passagens e, com isso, o tempo de execução. No entanto, ela é mais complexa de implementar e exige mais fitas para funcionar. Tal diferença entre os números de passadas pelas fitas é perceptível ao compararmos o número de transferências principalmente de leituras entre ambos os métodos, uma vez que com o arquivo desordenado, considerando o maior número de registros possível, a intercalação balanceada utilizando $f + 1$ fitas ultrapassa 3 milhões de transferências, enquanto a intercalação com $2f$ fitas sequer alcança esse número. Por outro lado, analisando o gasto com memória, a implementação com $f + 1$ fitas é mais vantajosa, pois tem um custo consideravelmente menor que a intercalação com $2f$ fitas.

Já o *QuickSort* externo é uma boa opção quando parte dos dados pode ser carregada na memória, pois é rápido e eficiente nesses casos. Porém, ele pode ter problemas se os dados não estiverem bem distribuídos, o que pode aumentar o tempo de execução e o número de comparações. Comparando com os métodos anteriores, o *QuickSort* externo é bastante extravagante em todos os aspectos (tempo, transferências, comparações). Além disso, durante o desenvolvimento foi encontrada uma dificuldade do computador ao lidar com a comparação entre pontos flutuantes ao utilizar o *QuickSort*, de maneira que as comparações de casas decimais começavam a ficar erradas a partir de um grande volume de dados. As chaves (notas dos alunos) precisaram ser multiplicadas por um múltiplo de 10 dentro das condicionais para garantir a exatidão da ordenação por completo. Apesar de esse provavelmente ser um problema do computador e não do algoritmo, por si só, não deixa de ser importante ressaltar que não houveram quaisquer problemas desse tipo com a intercalação balanceada, em geral.

Em termos de tempo de execução, a intercalação com $2f$ fitas geralmente é a mais rápida para grandes volumes de dados, enquanto a intercalação com $f + 1$ fitas é mais lenta, mas mais fácil de usar. O Quicksort externo pode ser rápido em cenários específicos, mas não é tão consistente quanto

os outros métodos, e possui "saltos" de tempo maiores com o aumento de dados. Ou seja, enquanto a intercalação balanceada (tanto com $2f$ quanto com $f + 1$ caminhos) não chegou a um tempo de execução maior que 3 segundos, o *QuickSort* externo ultrapassou esse tempo em quase três vezes em sua execução mais longa, para um mesmo volume de dados. Quanto ao número de comparações, a intercalação com $f + 1$ fitas costuma ser a mais eficiente, seguida pela intercalação com $2f$ fitas, enquanto o Quicksort externo pode variar bastante.

No final das contas, cada método tem seus pontos fortes e fracos. Se você precisa de uma solução simples e rápida para volumes menores de dados, a intercalação com $f + 1$ fitas é uma boa escolha. Para grandes volumes, a intercalação com $2f$ fitas é mais eficiente, apesar de ser mais complexa. Já o Quicksort externo pode ser útil em casos específicos, mas exige mais cuidado na implementação. A decisão final vai depender do tamanho dos dados, da complexidade que você pode lidar e do desempenho que você espera alcançar.